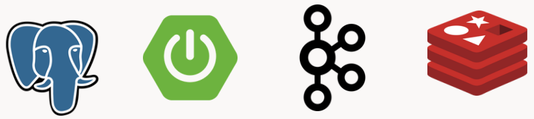


Spring Batch: xử lý hàng triệu bản ghi an toàn

Cách hoạt động và khi nào nên dùng trong thực tế



Cuối tháng, ngân hàng phải cộng lãi cho 5 triệu tài khoản

Nếu nghĩ đơn giản: chỉ cần lặp qua từng tài khoản, tính số tiền lãi dựa trên số dư, rồi cộng tiền vào tài khoản.

Dữ liệu cực lớn

- 5 triệu tài khoản đang hoạt động
- Kích thước nhiều Gigabyte
- Không thể tải hết vào bộ nhớ

Chạy định kỳ

- Mỗi tháng một lần vào nửa đêm
- Khung giờ chạy được lên lịch sẵn
- Không có con người ngồi bấm tay

Không trả lời ngay

- Chạy ngầm ở phía sau hệ thống
- Khách hàng không đứng đợi màn hình
- Ưu tiên độ chính xác tuyệt đối

Loại công việc xử lý dữ liệu lớn theo lô định kỳ này được gọi là Batch.

Vấn đề khi dùng vòng lặp

1. Quá tải bộ nhớ

Nạp 5 triệu tài khoản vào bộ nhớ RAM thì máy chủ sẽ lập tức bị sập (lỗi OutOfMemory).

2. Dữ liệu hỏng

Nếu có 1 tài khoản có số liệu lỗi, cả vòng lặp bị ngắt

3. Mất điện sự cố

Mất điện ở tài khoản 3 triệu, chạy lại từ đầu thì 3 triệu người đầu bị cộng lãi 2 lần.

4. Mù thông tin

Không biết đang chạy đến đâu, xong bao nhiêu, còn bao lâu và lỗi ở dòng nào.

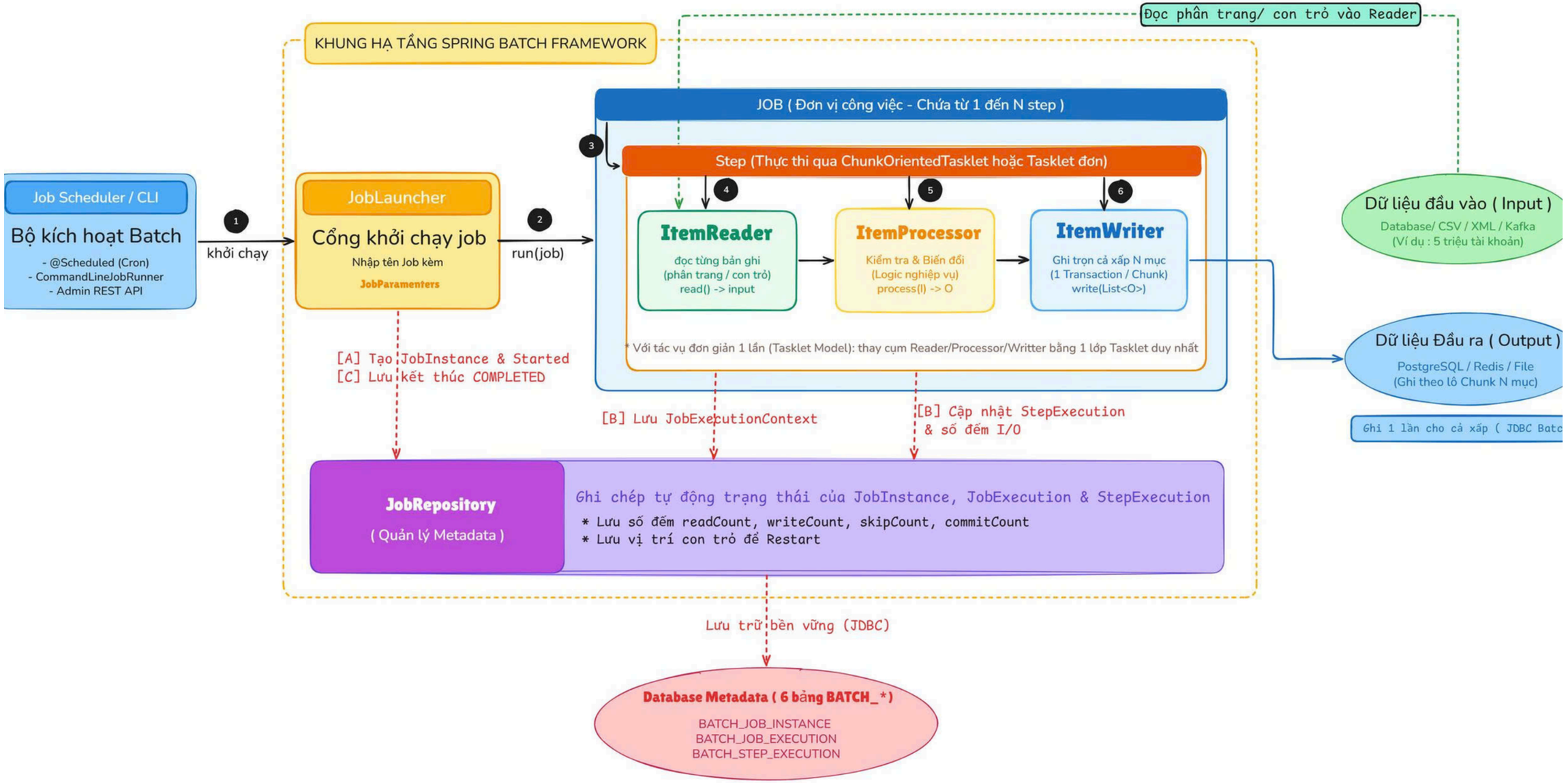
Vậy Spring Batch sửa painpoint này thế nào?

Spring Batch: bộ công cụ có sẵn cho đúng 4 rắc rối đó

Framework tiêu chuẩn của hệ sinh thái Spring chuyên biệt cho xử lý dữ liệu hàng loạt.

1. Nhiều dữ liệu (RAM quá tải)	→	Xử lý theo từng nhóm nhỏ (Chunk Processing)
2. Một bản ghi hỏng dừng cả đợt	→	Tự động bỏ qua và ghi nhật ký lỗi (Skip)
3. Mất điện sự cố giữa chừng	→	Ghi nhớ mốc an toàn, chạy tiếp mượt mà (Restart)
4. Không biết tiến độ và trạng thái	→	Sổ ghi chép chi tiết tự động vào database (JobRepository)

Kiến trúc tổng thể Spring Batch và 2 luồng xử lý



Dây chuyền 3 khâu: đọc, xử lý và ghi

Mô hình nhà máy đóng gói: nguyên liệu thô được đưa vào qua khâu Lấy hàng, chuyển sang khâu Kiểm tra dán nhãn, rồi đưa vào khâu Đóng thùng lưu kho.

ItemReader

- Lấy hàng từ kho
- Đọc từng tài khoản từ database
- Phân trang tự động theo từng khối

ItemProcessor

- Kiểm tra và dán nhãn
- Tính toán số tiền lãi theo công thức
- Loại bỏ bản ghi không hợp lệ

ItemWriter

- Xếp vào kho thành phẩm
- Cập nhật số dư mới vào database
- Ghi trọn vẹn cả lô trong 1 giao dịch

Step là một dây chuyền, Job là cả nhà máy

Nhà máy (Job) gồm nhiều dây chuyền (Step) chạy nối tiếp. Step trước hoàn tất thành công thì Step sau mới được kích hoạt để đảm bảo an toàn tuyệt đối.

JOB : TÍNH LÃI VÀ BÁO CÁO CUỐI THÁNG

Step 1: Tính lãi	Reader -> Processor -> Writer cho 5 triệu tài khoản
↓	
Step 2: Thông báo	Tạo thư thông báo biến động số dư cho khách hàng
↓	
Step 3: Báo cáo	Tổng hợp số liệu xuất báo cáo gửi ban giám đốc

Quy tắc an toàn: Nếu Step 1 bị lỗi, các Step sau sẽ không chạy tiếp để tránh gửi thông báo sai hoặc xuất báo cáo bẩn.

Chunk: chia để trị có chọn lọc

Ví dụ chấm thi: cô giáo chia 500 bài thi thành từng xấp 20 bài. Chấm xong mỗi xấp thì ghi điểm ngay vào sổ. Nếu có sự cố, cô chỉ cần chấm lại xấp đang dở, không phải chấm lại từ đầu.

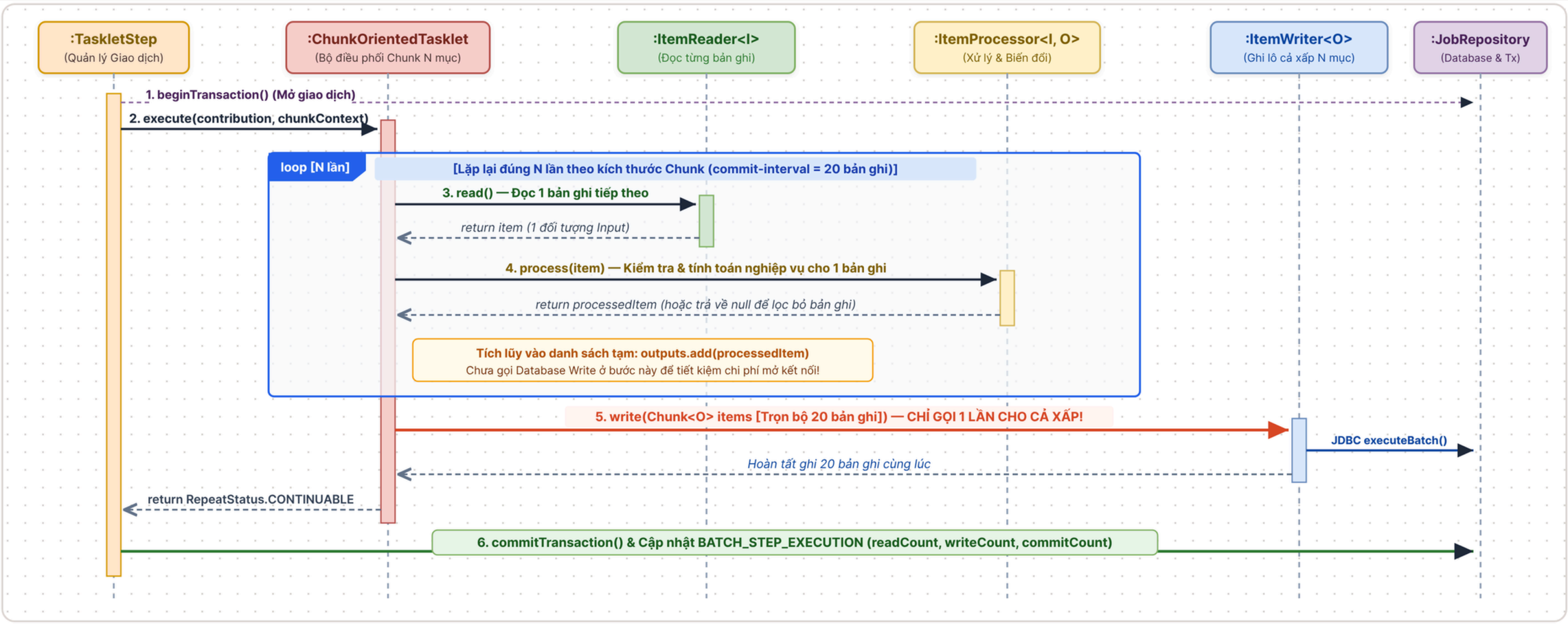
Cơ chế xử lý theo Chunk (N bản ghi):

- Reader đọc từng bản ghi cho đủ N mục
- Processor nhận từng mục và xử lý
- Writer nhận trọn vẹn N mục đã xử lý
- Commit một giao dịch duy nhất cho cả xấp

Lựa chọn kích thước Chunk phù hợp:

- Xấp quá nhỏ: lưu liên tục tốn công mở giao dịch
- Xấp quá lớn: tốn bộ nhớ RAM, nếu lỗi phải làm lại nhiều
- Thực tế: thường chọn từ 20 đến 500 bản ghi
- Mỗi xấp hoàn tất là một mốc an toàn được lưu chắc

Sơ đồ trình tự xử lý Chunk của ChunkOrientedTasklet



Cơ chế tối ưu I/O: ItemReader & ItemProcessor chạy lặp từng bản ghi (N lần), nhưng ItemWriter chỉ gọi 1 lần duy nhất cho cả xấp N mục.

Sổ ghi chép: Spring Batch nhớ mình đã làm tới đâu

Như cuốn sổ nhật ký công trường: Spring Batch tự động ghi chép chi tiết mỗi lần chạy, số lượng bản ghi đã đọc, đã ghi và trạng thái thành công hay lỗi.

Các khái niệm cốt lõi:

- **JobLauncher:** nút bấm khởi chạy công việc
- **JobRepository:** cuốn sổ ghi chép các bảng BATCH
- **JobInstance:** công việc cụ thể (ví dụ: Tính lãi tháng 10)
- **JobExecution:** mỗi lần bấm máy (chạy lại là một execution mới của cùng instance)

Trạng thái và số liệu theo dõi:

Trạng thái chuẩn: STARTED, COMPLETED, FAILED

readCount
tổng số bản ghi đã đọc từ database

writeCount
tổng số bản ghi đã ghi thành công

commitCount
số lần lưu giao dịch thành công

skipCount
số bản ghi lỗi được bỏ qua an toàn

Khi gặp sự cố: Skip, Retry và Restart

Skip (Bỏ qua)

- Một tài khoản bị lỗi dữ liệu
- Bỏ qua và ghi chép vào nhật ký
- Cả đợt 5 triệu người vẫn tiếp tục

Retry (Thử lại)

- Gọi dịch vụ ngoài bị nghẽn mạng
- Thử lại vài lần với độ trễ ngắn
- Tính năng có sẵn của framework

Restart (Chạy tiếp)

- Mất điện ở tài khoản thứ 3 triệu
- Khi bật lại, đọc mốc trong sổ
- Chạy tiếp từ xấp đã lưu chắc

Khung cảnh báo: Restart chỉ an toàn khi bước ghi không tạo trùng lặp dữ liệu (ghi theo khóa chính, kiểu cập nhật thay vì cộng dồn số học).

Bước 1: Cài đặt và khởi tạo dự án Spring Batch

```
<!-- 1. Dự án Maven chuẩn (Java 17+, Spring Batch 6.0.6) -->
<dependencies>
  <dependency>
    <groupId>org.springframework.batch</groupId>
    <artifactId>spring-batch-core</artifactId>
    <version>6.0.6</version>
  </dependency>
</dependencies>

<!-- 2. Dự án Spring Boot thực tế (Tự động cấu hình) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-batch</artifactId>
</dependency>
```

Yêu cầu môi trường (Java 17+ & Maven)

= Cần JDK 17+ để chạy ứng dụng (hoặc JDK 22+ & Docker khi build từ source bằng lệnh ./mvnw package)

spring-batch-core (Phiên bản 6.0.6)

= Thư viện cốt lõi cung cấp Job, Step, JobOperator và JobRepository cho dự án Spring chuẩn

spring-boot-starter-batch

= Khuyên dùng trong thực tế: tự động kết nối Database và tạo sẵn hệ thống bảng lưu vết BATCH_*

Tài nguyên & Dự án mẫu chính thức

= Thực hành ETL với Creating a Batch Service và kho mẫu Spring Batch Samples

Phiên bản mới nhất (10/2026): Spring Batch 6.0.6 và 6.1.0-RC1 (cùng nhánh 5.2.6) đã phát hành trên Maven Central.

Bước 2: Cấu hình Job, Step và ItemProcessor

```java
// 1. Cấu hình Job & Step cơ bản (Getting Started Tutorial)
@Configuration  @EnableBatchProcessing
public class HelloWorldJobConfiguration {
    @Bean
    public Step step(JobRepository jobRepository) {
        return new StepBuilder(jobRepository).tasklet((contribution, chunkContext) -> {
            System.out.println("Hello world!"); return RepeatStatus.FINISHED;
        }).build();
    }
    @Bean
    public Job job(JobRepository jobRepository, Step step) {
        return new JobBuilder(jobRepository).start(step).build();
    }
}
```

```java
// 2. Mẫu cài đặt ItemProcessor<I, O> chuẩn (Terasoluna Guideline)
public class MyItemProcessor implements ItemProcessor<MyInput, MyOutput> { // (1)
    @Override
    public MyOutput process(MyInput item) throws Exception {      // (2)
        MyOutput out = new MyOutput(); /* Xử lý nghiệp vụ */  // (3)
        return out;                                           // (4)
    }
}
```

@Configuration & @EnableBatchProcessing

= Kích hoạt hạ tầng Spring Batch, tự động tạo sẵn JobRepository, JobOperator và quản lý giao dịch

StepBuilder & JobBuilder

= Khởi tạo dây chuyền Step (.tasklet hoặc .chunk) và đóng gói vào nhà máy Job (.start(step))

ItemProcessor<I, O> (Bước 1 & 2)

= Chỉ định kiểu dữ liệu Đầu vào / Đầu ra bằng Generics an toàn và nhận từng bản ghi qua hàm process(item)

Biến đổi hoặc Lọc bỏ bản ghi (Bước 3 & 4)

= Khởi tạo đối tượng đầu ra chứa kết quả nghiệp vụ (trả về null nếu muốn lọc bỏ bản ghi khỏi ItemWriter)

Tùy chọn lược bỏ ItemProcessor

= Nếu chỉ đọc và ghi thẳng không cần biến đổi, có thể lược bỏ Processor (tương đương PassThroughItemProcessor)

Điểm mới Spring Batch 6 & Terasoluna: StepBuilder và ItemProcessor<I, O> giúp tách biệt logic nghiệp vụ khỏi tầng đọc/ghi.

Bước 3: Khởi chạy Job và đọc nhật ký thực thi

```
// Cách 1: Khởi chạy trong Java bằng JobOperator (Spring Batch 6)
ApplicationContext ctx = new
AnnotationConfigApplicationContext(HelloWorldJobConfiguration.class);
JobOperator jobOperator = ctx.getBean(JobOperator.class);
jobOperator.start(ctx.getBean(Job.class), new JobParameters());

// Cách 2: Khởi chạy từ dòng lệnh với CommandLineJobRunner (Terasoluna)
$ java org.springframework.batch.core.launch.support.CommandLineJobRunner \
      <jobPath> <jobName> targetMonth=2026-10
```

```
// Nhật ký vận hành thực tế (Standard Output Log)
[main] INFO ...JobLauncher - Job: [SimpleJob: [name=job]] launched with parameters: [{}]
[main] INFO ...AbstractStep - Executing step: [step]
Hello world!
[main] INFO ...AbstractStep - Step: [step] executed in 3ms
[main] INFO ...JobLauncher - Job completed with status: [COMPLETED] in 4ms
```

1. AnnotationConfigApplicationContext

= Khởi tạo Spring IoC Container từ class cấu hình để lấy các Bean quản lý hạ tầng Batch

2. jobOperator.start(job, new JobParameters())

= Kích hoạt chạy Job trong mã nguồn Java kèm bộ tham số định danh cho từng đợt chạy

3. CommandLineJobRunner (Chạy qua Shell/Cron)

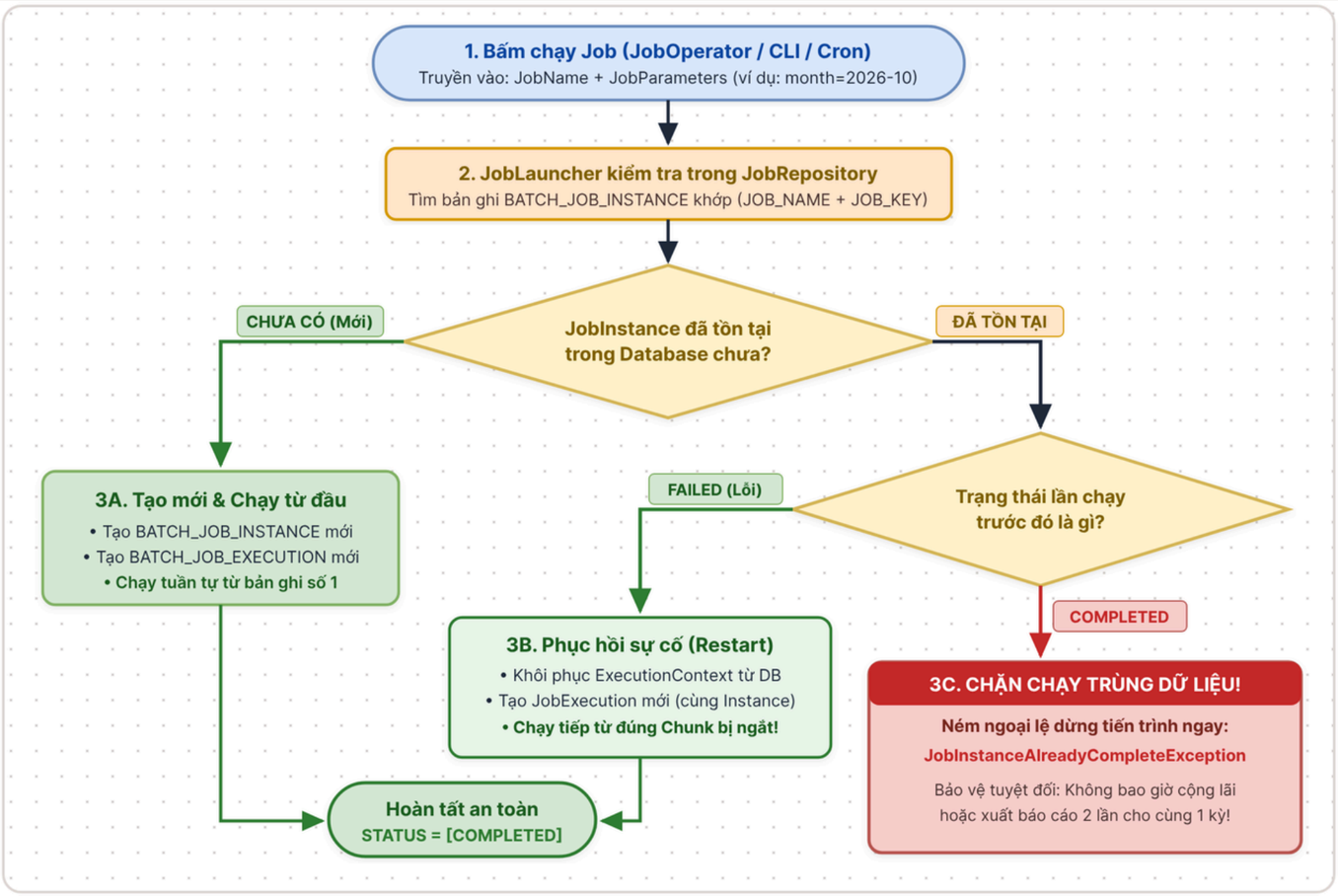
= Tiện ích có sẵn của Spring Batch giúp gọi Job trực tiếp từ lệnh java kèm các đối số key=value

4. Theo dõi nhật ký thực thi [COMPLETED]

= Ghi nhận rõ thời gian chạy từng Step (3ms), tổng thời gian Job (4ms) và trạng thái kết thúc

Hỗ trợ chính thức (từ 01/2026): GitHub Discussions cho hỏi đáp & GitHub Issues cho báo lỗi (ngừng hỗ trợ StackOverflow).

Lưu đồ khởi chạy Job và cơ chế chống chạy trùng / Restart



Công thức định danh: JobInstance = JobName + JobKey

= Mỗi cặp (Tên Job + Bộ tham số JobParameters) tạo ra 1 JobInstance duy nhất trong bảng BATCH_JOB_INSTANCE

Nhánh 3A (Chưa có) — Chạy mới từ đầu

= Khi truyền tham số mới (ví dụ: month=2026-11), hệ thống tạo JobInstance mới và xử lý từ bản ghi số 1

Nhánh 3B (FAILED) — Phục hồi sự cố (Restart)

= Nếu lần chạy trước bị mất điện giữa chừng, truyền lại đúng tham số cũ sẽ khôi phục Context để chạy tiếp

Nhánh 3C (COMPLETED) — Chặn chạy trùng

= Nếu kỳ đó đã COMPLETED mà bấm chạy lại cùng tham số, hệ thống ném lỗi ngay để tránh cộng tiền 2 lần

Bước 4: Mã nguồn triển khai thực tế (Chunk & Skip)

```
@Bean
public Step trendingStep(JobRepository repo, TxManager tx) {
        return new StepBuilder("trendingPostsStep", repo).chunk(20, tx)
       .reader(reader()).processor(proc).writer(writer)
       .faultTolerant().skip(MalformedException.class)
       .skipLimit(100).build();
}

@Bean
public Job trendingJob(JobRepository repo, Step step) {
         return new JobBuilder("trendingPostsJob", repo)
        .start(step).build();
}
```

.chunk(20, tx)

= Xử lý theo nhóm 20 bản ghi một giao dịch (Slide 8–9)

reader / processor / writer

= Dây chuyền chuẩn hóa 3 khâu đọc, xử lý và ghi (Slide 6)

.faultTolerant() .skip()

= Bật chịu lỗi, tự bỏ qua dữ liệu hỏng tối đa 100 lần (Slide 12)

StepBuilder / JobBuilder

= Khai báo Step dây chuyền và Job cả nhà máy (Slide 7)

repo (JobRepository)

= Tự động ghi chép tiến độ vào 6 bảng database (Slide 10–11)

Khi nào nên dùng Spring Batch, khi nào không nên

NÊN DÙNG (RẤT PHÙ HỢP)

- Tính toán định kỳ hàng triệu bản ghi (tính lãi, tính lương)
- Gửi email hoặc thông báo hàng loạt cho người dùng
- Tổng hợp báo cáo tài chính và thống kê cuối ngày
- Đồng bộ, chuyển đổi dữ liệu giữa các hệ thống
- Tính trước điểm gợi ý hoặc bảng xếp hạng nội dung

KHÔNG NÊN DÙNG

- Cần trả lời ngay khi người dùng bấm nút (nhắn tin, đăng nhập)
- Xử lý luồng sự kiện liên tục theo thời gian thực (streaming)
- Việc nhỏ vài trăm bản ghi (chỉ cần tác vụ định kỳ đơn giản)
- Công việc không cần ghi nhớ tiến độ hoặc phục hồi sự cố

Quy tắc ngón tay cái: Dữ liệu lớn, chạy định kỳ, không cần phản hồi tức thì, cần chịu lỗi và theo dõi chặt chẽ.

Tóm tắt: 5 điều cần nhớ về Spring Batch

Job và Step

Cấu trúc nhà máy

Job gồm nhiều Step chạy nối tiếp theo thứ tự an toàn

3 Khâu Chuẩn

Reader, Proc, Writer

Đọc bản ghi, xử lý biến đổi và ghi theo lô

Chunk

Chia nhỏ dữ liệu

Xử lý theo từng xấp nhỏ, commit từng giao dịch chắc chắn

Sổ Ghi Chép

JobRepository

Tự động ghi nhớ trạng thái và số liệu vào database

Chịu Lỗi

Skip, Restart

Bỏ qua bản ghi hỏng và chạy tiếp khi gặp sự cố

DEMO THỰC TẾ